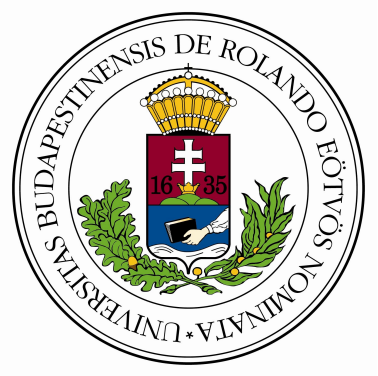
# Konkurens programozás

## Szálak és feladatok




**Kozsik Tamás**

ELTE Eötvös Loránd Tudományegyetem

# Outline


1. Problémafelvetés
   - Rendezés párhuzamosan
2. Executor
3. ExecutorService
   - Rendezés ismét
4. Future
5. ForkJoinPool

# Példa: sok nagy szám faktorizálása

```
ArrayList<BigInteger>[] factorizeAll( BigInteger[] inputs ){
    ...
}

ArrayList<BigInteger> factorize( BigInteger n ){
    ...
}
```



# Általánosítás: sok adatra végrehajtott bonyolult számítás

```
Output[] computeAll( Input[] inputs ){
    Output[] outputs = new Output[inputs.length];
    for( int i=0; i<inputs.length; ++i ){
        outputs[i] = computeOne(inputs[i]);
    }
    return outputs;
}

Output computeOne( Input input ){ ... }
```



# Párhuzamosítás

```
Output[] computeAll( Input[] inputs ) throws InterruptedException {
    Output[] outputs = new Output[inputs.length];
    Thread[] threads = new Thread[inputs.length];
    for( int i=0; i<inputs.length; ++i ){
        int I = i;
        threads[i] = new Thread( () ->
                                   outputs[I] = computeOne(inputs[I])
                                 );
        threads[i].start();
    }
    for( int i=0; i<inputs.length; ++i ){ threads[i].join(); }
    return outputs;
}

Output computeOne( Input input ){ ... }
```



# Párhuzamosítás (alternatív megfogalmazás)

```
Output[] computeAll( Input[] inputs ) throws InterruptedException {
    Output[] outputs = new Output[inputs.length];
    Thread[] threads = new Thread[inputs.length];
    Arrays.setAll( threads, i -> new Thread( () ->
                                    outputs[i] = computeOne(inputs[i])
                                )
    );
    for( int i=0; i<inputs.length; ++i ){ threads[i].start(); }
    for( int i=0; i<inputs.length; ++i ){ threads[i].join();  }
    return outputs;
}

Output computeOne( Input input ){ ... }
```



# Miért helyes?

Módosuló állapotot osztunk meg szálak között!



# Miért helyes?

Módosuló állapotot osztunk meg szálak között!

- A szálindítás szinkronizációs esemény
- A `join()`-olás szinkronizációs esemény



# Rendezés lineáris időben lineáris számú processzorral

**input**

alma
a
fa
alatt



# Rendezés lineáris időben lineáris számú processzorral

| input |
| --- |
| alma |
| a |
| fa |
| alatt |

| indexek |
| --- |
| 2 |
| 0 |
| 3 |
| 1 |



# Rendezés lineáris időben lineáris számú processzorral

| input |
| --- |
| alma |
| a |
| fa |
| alatt |

| indexek |
| --- |
| 2 |
| 0 |
| 3 |
| 1 |

| output |
| --- |
| a |
| alatt |
| alma |
| fa |



# Egy tehát egy hasonló példa

```
String[] sort( String[] inputs ) throws InterruptedException {
    String[] outputs = new String[inputs.length];
    Thread[] threads = new Thread[inputs.length];
    for( int i=0; i<inputs.length; ++i ){
        final int I = i;
        threads[i] = new Thread( () -> arrangeOne(I,inputs,outputs) );
        threads[i].start();
    }
    for( int i=0; i<inputs.length; ++i ){ threads[i].join();  }
    return outputs;
}
```



# arrangeOne

```
void arrangeOne( int i, String[] inputs, String[] outputs ){
    int index = 0;
    for( int j=0; j<i; ++j ){
        if( inputs[j].compareTo(inputs[i]) <= 0 ){
            ++index;
        }
    }
    for( int j=i+1; j<inputs.length; ++j ){
        if( inputs[j].compareTo(inputs[i]) < 0 ){
            ++index;
        }
    }
    outputs[index] = inputs[i];
}
```



# Mi ezzel a probléma?

- Csak akkor jó, ha rengeteg processzormagunk van
- Túl sok szál indítása magas overheaddel jár



# Mi ezzel a probléma?

- Csak akkor jó, ha rengeteg processzormagunk van
- Túl sok szál indítása magas overheaddel jár

## Megoldás

Indítsunk annyi szálat, ahány processzormagunk van, egy szálra sok adatot bízzunk rá!



# Mi ezzel a probléma?

- Csak akkor jó, ha rengeteg processzormagunk van
- Túl sok szál indítása magas overheaddel jár

## Megoldás

Indítsunk annyi szálat, ahány processzormagunk van, egy szálra sok adatot bízzunk rá!

```
Runtime.getRuntime().availableProcessors()
```



# Előre megadott számú szállal

```
String[] sort( String[] inputs, int parallelism ) throws InterruptedException {
    String[] outputs = new String[inputs.length];
    Thread[] threads = new Thread[parallelism];
    int itemsPerThread = inputs.length / parallelism;
    int remainder = inputs.length % parallelism;
    int startPos = 0;
    for( int i=0; i<threads.length; ++i ){
        final int S = startPos;
        final int todo = itemsPerThread + (i < remainder ? 1 : 0 );
        threads[i] = new Thread( () -> arrangeMany(S,todo,inputs,outputs) );
        threads[i].start();
        startPos += todo;
    }
    for( int i=0; i<threads.length; ++i ){ threads[i].join(); }
    return outputs;
}
```



# Előre megadott számú szállal

```
String[] sort( String[] inputs, int parallelism ) throws InterruptedException {
    String[] outputs = new String[inputs.length];
    Thread[] threads = new Thread[parallelism];
    int itemsPerThread = inputs.length / parallelism;
    int remainder = inputs.length % parallelism;
    int startPos = 0;
    for( int i=0; i<threads.length; ++i ){
        final int S = startPos;
        final int todo = itemsPerThread + (i < remainder ? 1 : 0 );
        threads[i] = new Thread( () -> arrangeMany(S,todo,inputs,outputs) );
        threads[i].start();
        startPos += todo;
    }
    for( int i=0; i<threads.length; ++i ){ threads[i].join(); }
    return outputs;
}
```

Ez azért már elég bonyolult kezd lenni...



# arrangeMany

```
void arrangeMany( int startPos, int todo, String[] inputs, String[] outputs ){

    while( todo > 0 ){
        --todo;
        arrangeOne(startPos+todo,inputs,outputs);
    }

}
```



# Mire kellett itt vigyázni?

Ugyanannyi munkát adjunk a szálaknak!



# Mire kellett itt vigyázni?

Ugyanannyi munkát adjunk a szálaknak!

Mindenkinek `itemsPerThread` számú, egyenlő nehézségű részfeladat, néhánynak eggyel több



# Mire kellett itt vigyázni?

Ugyanannyi munkát adjunk a szálaknak!

Mindenkinek `itemsPerThread` számú, egyenlő nehézségű részfeladat, néhánynak eggyel több

És ha a részfeladatok különböző nehézségűek? Pl. faktorizáció?



# Mire kellett itt vigyázni?

Ugyanannyi munkát adjunk a szálaknak!

Mindenkinek `itemsPerThread` számú, egyenlő nehézségű részfeladat, néhánynak eggyel több

És ha a részfeladatok különböző nehézségűek? Pl. faktorizáció?

Megoldandó feladatok és számítást végző szálak szétválasztása!



# Outline


1. Problémafelvetés
   - Rendezés párhuzamosan
2. Executor
3. ExecutorService
   - Rendezés ismét
4. Future
5. ForkJoinPool

# Feladat

- Az elvégzendő munka egy egysége
- Konkurrens végrehajtás szálakban
- A feladatok méretének megválasztása kritikus
  - Feladat = szál költséges lehet
  - Túl kicsi feladatok: versengés, ütemezési overhead
  - Túl nagy feladatok: kismértékű konkurrencia
- Feladatok és szálak explicit összekapcsolása: skálázási problémák



# Executor interface

## java.lang.Runnable

```
void run()
```

## java.util.concurrent.Executor

```
void execute(Runnable) throws RejectedExecutionException
```



# Megvalósítás: Executors factory osztály

- `newCachedThreadPool()`
- `newFixedThreadPool(int nThreads)`
- `newSingleThreadExecutor()`



# Szekvenciális szerver (JCIP)

```
import java.io.IOException;
import java.net.*;

public class SingleThreadWebServer {
    public static void main(String[] args) throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            Socket connection = socket.accept();
            handleRequest(connection);
        }
    }

    private static void handleRequest(Socket connection) {
        // request-handling logic here
    }
}
```



# Kliensenként új szál (JCIP)

```
public class ThreadPerTaskWebServer {
    public static void main(String[] args) throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            final Socket connection = socket.accept();
            Runnable task = new Runnable() {
                public void run() {
                    handleRequest(connection);
                }
            };
            new Thread(task).start();
        }
    }

    private static void handleRequest(Socket connection) {
        // request-handling logic here
    }
}
```



# Kliensenként egy feladat (JCIP)

```
public class TaskExecutionWebServer {
    private static final int NTHREADS = 100;
    private static final Executor exec
                        = Executors.newFixedThreadPool(NTHREADS);

    public static void main(String[] args) throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            final Socket connection = socket.accept();
            Runnable task = new Runnable() {
                public void run() {
                    handleRequest(connection);
                }
            };
            exec.execute(task);
        }
    }
...
```



# Kliensenként egy feladat

```
public class TaskExecutionWebServer {
    private static final int NTHREADS = 100;
    private static final Executor exec
                        = Executors.newFixedThreadPool(NTHREADS);

    public static void main(String[] args) throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            final Socket connection = socket.accept();
            exec.execute( () -> handleRequest(connection) );
        }
    }

    private static void handleRequest(Socket connection) {
        // request-handling logic here
    }
}
```



# Outline


1. Problémafelvetés
   - Rendezés párhuzamosan
2. Executor
3. ExecutorService
   - Rendezés ismét
4. Future
5. ForkJoinPool

# Életcikluskezelés

## java.util.concurrent.Executor

```
void execute(Runnable) throws RejectedExecutionException
```

## java.util.concurrent.ExecutorService extends Executor

```
void shutdown()
List<Runnable> shutdownNow()
boolean isShutdown()
boolean isTerminated()
boolean awaitTermination(long, TimeUnit) throws InterruptedExc.
```



# Szerver életcikluskezeléssel (JCIP)

```
public class LifecycleWebServer {
    private final ExecutorService exec = Executors.newCachedThreadPool();

    public void start() throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (!exec.isShutdown()) {
            try {
                final Socket connection = socket.accept();
                exec.execute( () -> handleRequest(connection) );
            } catch (RejectedExecutionException e) {
                if (!exec.isShutdown())
                    log("task submission rejected", e);
            }
        }
    }

    public void stop() {
        exec.shutdown();
    }
...
```



# Feladat kiosztása

## java.util.concurrent.Executor

```
void execute(Runnable) throws RejectedExecutionException
```

## java.util.concurrent.ExecutorService extends Executor

```
<T>       Future<T> submit   (Callable<T>)
          Future<?> submit   (Runnable)
<T>       Future<T> submit   (Runnable, T)
<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>>)
<T>               T invokeAny(Collection<? extends Callable<T>>)
```



# Aszinkron elvégzett számítás eredménye

**java.lang.Runnable**

```
void run()
```

**java.util.concurrent.Callable<V>**

```
V call() throws java.lang.Exception
```

**java.util.concurrent.Future<V>**

```
V get() throws InterruptedException, ExecutionException
...
```

implementáció: `FutureTask<V>(Callable<V>)`



```
String[] sort( String[] inputs, ExecutorService exec ) throws Exception {
    String[] outputs = new String[inputs.length];
    Future<Integer>[] pending = new Future[inputs.length];
    Arrays.setAll( pending, i -> exec.submit( () -> smallerOnes(i,inputs)) );
    for( int i=0; i<inputs.length; ++i ){
        outputs[pending[i].get()] = inputs[i];
    }
    return outputs;
}
```



```
int smallerOnes( int i, String[] inputs ){
    int index = 0;
    for( int j=0; j<i; ++j )
        if( inputs[j].compareTo(inputs[i]) <= 0 ) ++index;
    for( int j=i+1; j<inputs.length; ++j )
        if( inputs[j].compareTo(inputs[i]) < 0 ) ++index;
    return index;
}
```



```
public static void main( String[] args ) throws Exception {
    int par = Runtime.getRuntime().availableProcessors();
    ExecutorService exec = Executors.newFixedThreadPool(par);
    for( String s: new Sorter.sort(args, exec) ){
        System.out.println(s);
    }
    exec.shutdown();
}
```



# Outline


1. Problémafelvetés
   - Rendezés párhuzamosan
2. Executor
3. ExecutorService
   - Rendezés ismét
4. Future
5. ForkJoinPool

# Future interfész leegyszerűsítve

```
public interface Future<V> {
    V get() throws InterruptedException, ExecutionException;
    boolean isDone();
}
```



# Future interfész leegyszerűsítve

```
public interface Future<V> {
    V get() throws InterruptedException, ExecutionException;
    boolean isDone();
}
```

A `java.util.concurrent.Future` további műveleteket is tartalmaz.



# Használat

```
public static void main( String[] args ) throws InterruptedException {
  Future[] futures = new Future[args.length];
  Arrays.setAll( futures,
                 i -> Async.call( () -> {
                                    int n = Integer.parseInt(args[i]);
                                    Thread.sleep(n);
                                    return null;
                                  }
                                )
               );
   for( Future future: futures ){
      try { future.get(); }
      catch( ExecutionException e ){
         e.getCause().printStackTrace();
      }
   }
}
```



# Future létrehozása: pl. aszinkron hívás

```
public class Async {

    public static <T> Future<T> call( Callable<T> task ){
        Promise<T> promise = new Promise<T>(task);
        promise.start();
        return promise;
    }
```



# Future létrehozása: pl. aszinkron hívás

```
public class Async {

    public static <T> Future<T> call( Callable<T> task ){
        Promise<T> promise = new Promise<T>(task);
        promise.start();
        return promise;
    }

    private static class Promise<T> extends Thread
                                    implements Future<T> {
        ...
    }

}
```



# Future egy lehetséges megvalósítása

```
class Promise<T> extends Thread implements Future<T> {
  private Callable<T> task;          private T result = null;
                                     private Throwable exc = null;
  Promise( Callable<T> task ){ this.task = task; }

  @Override public void run(){
      try { result = task.call(); }
      catch( Throwable t ){ exc = t; }
  }
  public T get() throws InterruptedException, ExecutionException {
      this.join();
      if( exc == null ){ return result; }
      else { throw new ExecutionException(exc); }
  }
  public boolean isDone(){ return isAlive(); }
}
```



# Outline


1. Problémafelvetés
   - Rendezés párhuzamosan
2. Executor
3. ExecutorService
   - Rendezés ismét
4. Future
5. ForkJoinPool

# Párhuzamos számítások: `ForkJoinTask`

- Tiszta számítás vagy izolált adat manipulálása
- Nagy tömegben használható
  - fine grained
  - 1000 számítási lépés
- `ForkJoinPool`ban futnak
- `fork`, `join`, `Future`, `invokeAll`
- Egy taszkból elforkolt taszkok ugyanabban a poolban
- Ne blokkolódjon (szinkronizáció, IO)

